

Lab 12 Remote push Notification P2

We are building a mobile app that connects to a Node.js server over WebSocket.
The server sends your phone a push notification **every 2 minutes**, automatically, as long as it is connected.

The flow:

1. App asks for notification permission and gets an Expo push token
2. App sends that token to the server over WebSocket
3. Server starts a timer and calls the Expo Push API every 2 minutes
4. Expo delivers the notification to your phone

Prerequisites

- Lab 11

Folder structure

```
your_app/
├── app/
│   ├── _layout.tsx
│   └── index.tsx
├── hooks/
│   └── usePushNotifications.ts
└── app.json
server/
├── server.js
├── package.json
└── .env
```

PROF

Setup

Mobile app

create the mobile app with the following packages

```
npx create-expo-app@latest ...
npx expo install expo-notifications expo-device expo-constants
npm run reset-project
```

and create the files to match the structure above.

Server

```
mkdir server && cd server
npm init -y
npm install ws expo-server-sdk dotenv
```

Mobile App

mobile/app.json

Replace `YOUR_EAS_PROJECT_ID` with your project ID from the setup video.

```
{
  "expo": {
    "name": "...",
    "slug": "...",
    //....
    "extra": {
      "eas": {
        "projectId": "YOUR_EAS_PROJECT_ID"
      }
    }
  }
}
```

mobile/app/_layout.tsx

Replace whatever `reset-project` left here with this:

```
import { Stack } from "expo-router";

export default function RootLayout() {
  return (
    <Stack>
      <Stack.Screen name="index" options={{ title: "🔔 Push Demo" }} />
    </Stack>
  );
}
```

mobile/app/index.tsx

```
import React, { useEffect, useRef, useState } from "react";
import { View, Text, StyleSheet, ScrollView } from "react-native";
import { usePushNotifications } from "../hooks/usePushNotifications";

// change this to your machine's local IP, e.g. ws://192.168.1.42:3001
```

```

// Mac/Linux: ifconfig | grep "inet "    Windows: ipconfig
const WS_URL = "ws://YOUR_LOCAL_IP:3001";

type LogEntry = { id: number; text: string };

export default function HomeScreen() {
  const { expoPushToken, lastNotification } = usePushNotifications();
  const ws = useRef<WebSocket | null>(null);

  const [connected, setConnected] = useState(false);
  const [notifCount, setNotifCount] = useState(0);
  const [logs, setLogs] = useState<LogEntry[]>([]);
  const logIdRef = useRef(0);

  const addLog = (text: string) => {
    logIdRef.current += 1;
    const id = logIdRef.current;
    setLogs((prev) => [
      { id, text: `[${new Date().toLocaleTimeString()}] ${text}` },
      ...prev.slice(0, 29),
    ]);
  };

  // — TODO 2

  // Open the WebSocket and send the push token to the server.
  //
  // This runs every time expoPushToken changes – it's null on the first
  render
  // so we wait until it's ready before connecting.
  //
  // Steps:
  //   1. if expoPushToken is null → return early
  //   2. const socket = new WebSocket(WS_URL)
  //   3. socket.onopen →
  //       send: socket.send(JSON.stringify({ type: "register",
  pushToken: expoPushToken })))
  //       setConnected(true)
  //       addLog("connected – token sent to server")
  //   4. socket.onmessage →
  //       const data = JSON.parse(event.data)
  //       if data.type === "ack" → addLog(`server: ${data.message}`)
  //   5. socket.onclose → setConnected(false), addLog("disconnected")
  //   6. socket.onerror → addLog("connection error"),
  setConnected(false)
  //   7. ws.current = socket
  //   8. return () => ws.current?.close()
  //
  useEffect(() => {
    // your code here
  }, [expoPushToken]);

  // — TODO 3

```

```

// React when a new notification arrives.
//
// lastNotification comes from your hook (same listener you wrote in
TODO 1).
// It updates every time a push notification is received.
//
// Steps:
// 1. if lastNotification is null → return early
// 2. setNotifCount(prev => prev + 1)
// 3. const title = lastNotification.request.content.title ?? "no
title"
//      addLog(`got notification: "${title}"`)
//
useEffect(() => {
  // your code here
}, [lastNotification]);

return (
  <View style={styles.container}>
    <Text style={styles.title}>🔔 Push Demo</Text>

    {/* connection badge */}
    <View style={[styles.badge, connected ? styles.green :
styles.red]}>
      <Text style={styles.badgeText}>
        {connected ? "🟢 Connected to server" : "🔴 Not connected"}
      </Text>
    </View>

    {/* push token */}
    <View style={styles.card}>
      <Text style={styles.label}>Your Push Token</Text>
      <Text style={styles.token} numberOfLines={3}>
        {expoPushToken ?? "getting token..."}
      </Text>
    </View>

    {/* notification counter */}
    <View style={styles.card}>
      <Text style={styles.label}>Notifications received this
session</Text>
      <Text style={styles.counter}>{notifCount}</Text>
      <Text style={styles.hint}>
        The server sends one every 2 minutes while you're connected.
      </Text>
    </View>

    {/* last notification content */}
    {lastNotification && (
      <View style={styles.notifCard}>
        <Text style={styles.notifTitle}>
          {lastNotification.request.content.title ?? "notification"}
        </Text>
      </View>
    )}
  </View>
)

```

```

        </Text>
        <Text style={styles.notifBody}>
            {lastNotification.request.content.body ?? ""}
        </Text>
    </View>
  )}

  { /* live log panel */ }
  <Text style={styles.label}>Log</Text>
  <ScrollView style={styles.logBox}>
    {logs.map((entry) => (
      <Text key={entry.id} style={styles.logLine}>
        {entry.text}
      </Text>
    ))}
  </ScrollView>
</View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#0F172A", padding: 16 },
  title: { color: "#F8FAFC", fontSize: 22, fontWeight: "700",
marginBottom: 12 },
  badge: {
    alignSelf: "flex-start",
    paddingHorizontal: 12,
    paddingVertical: 4,
    borderRadius: 99,
    marginBottom: 12,
  },
  green: { backgroundColor: "#16A34A22" },
  red: { backgroundColor: "#DC262622" },
  badgeText: { color: "#F8FAFC", fontSize: 13 },
  card: {
    backgroundColor: "#1E293B",
    borderRadius: 12,
    padding: 14,
    marginBottom: 12,
  },
  label: {
    color: "#94A3B8",
    fontSize: 12,
    fontWeight: "600",
    marginBottom: 4,
    textTransform: "uppercase",
  },
  token: { color: "#6366F1", fontSize: 11, fontFamily: "monospace" },
  counter: {
    color: "#F8FAFC",
    fontSize: 48,
    fontWeight: "800",
    textAlign: "center",

```

```

        paddingVertical: 8,
    },
    hint: { color: "#64748B", fontSize: 12, textAlign: "center" },
    notifCard: {
        backgroundColor: "#6366F120",
        borderLeftWidth: 4,
        borderLeftColor: "#6366F1",
        borderRadius: 8,
        padding: 12,
        marginBottom: 12,
    },
    notifTitle: { color: "#6366F1", fontWeight: "700", fontSize: 14 },
    notifBody: { color: "#CBD5E1", fontSize: 13, marginTop: 4 },
    logBox: {
        flex: 1,
        backgroundColor: "#0D1117",
        borderRadius: 8,
        padding: 8,
    },
    logLine: {
        color: "#4ADE80",
        fontSize: 11,
        fontFamily: "monospace",
        marginBottom: 2,
    },
});

```

Server Code

server/package.json

```

{
  "name": "push-demo-server",
  "version": "1.0.0",
  "type": "commonjs",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js"
  },
  "dependencies": {
    "dotenv": "latest",
    "expo-server-sdk": "latest",
    "ws": "latest"
  }
}

```

server/.env

```
PORT=3001
INTERVAL_MINUTES=2
# EXPO_ACCESS_TOKEN=your_token_here ← only needed if you enabled push
security
```

server/server.js

```
require("dotenv").config();
const { WebSocketServer, WebSocket } = require("ws");
const { Expo } = require("expo-server-sdk");

const PORT = parseInt(process.env.PORT ?? "3001", 10);
const INTERVAL_MS = parseFloat(process.env.INTERVAL_MINUTES ?? "2") * 60
  * 1000;

// WebSocket is imported so we can use WebSocket.OPEN (=== 1) as a
// constant
// without it, socket.OPEN would be undefined and the readyState check
// would break

const expo = new Expo({
  // accessToken: process.env.EXPO_ACCESS_TOKEN,
});

// socket → { pushToken, timer }
const clients = new Map();

async function sendPushNotification(pushToken) {
  if (!Expo.isExpoPushToken(pushToken)) {
    console.error("invalid push token:", pushToken);
    return;
  }
}

const messages = [
  {
    to: pushToken,
    sound: "default",
    title: "👋 Hello from the server!",
    body: `Sent at ${new Date().toLocaleTimeString()}`,
    channelId: "default", // must match the Android channel name
    created in the app
  },
];

// chunkPushNotifications groups messages into batches of 100 max
// we only have one here, but this is the right pattern
const chunks = expo.chunkPushNotifications(messages);

for (const chunk of chunks) {
```

```

    try {
      const tickets = await expo.sendPushNotificationsAsync(chunk);

      tickets.forEach((ticket) => {
        if (ticket.status === "ok") {
          // ticket.id lets you check the receipt later to confirm
          delivery
            console.log("push sent, receipt id:", ticket.id);
        } else {
          console.error("push error:", ticket.message,
            ticket.details?.error);
        }
      });
    } catch (err) {
      console.error("failed to send chunk:", err.message);
    }
  }
}

function startPushInterval(socket, pushToken) {
  console.log(
    `starting interval (every ${INTERVAL_MS / 1000}s) for:
    ${pushToken.slice(0, 30)}...`
  );

  // send the first one right away so you don't wait 2 minutes to verify
  it works
  sendPushNotification(pushToken);

  const timer = setInterval(() => {
    if (socket.readyState !== WebSocket.OPEN) {
      clearInterval(timer);
      return;
    }
    sendPushNotification(pushToken);
  }, INTERVAL_MS);

  return timer;
}

const wss = new WebSocketServer({ port: PORT });
console.log(`server running on ws://localhost:${PORT}`);
console.log(`push interval: every ${INTERVAL_MS / 1000}s`);

wss.on("connection", (socket) => {
  console.log("new client connected");

  socket.on("message", (raw) => {
    let data;
    try {
      data = JSON.parse(raw.toString());
    } catch {
      console.error("could not parse message:", raw.toString());
    }
  });
});

```



```

    return;
  }

  if (data.type === "register") {
    const { pushToken } = data;
    if (!pushToken) {
      console.warn("register: missing pushToken");
      return;
    }

    // clear any existing timer if the client re-registers
    const existing = clients.get(socket);
    if (existing?.timer) clearInterval(existing.timer);

    const timer = startPushInterval(socket, pushToken);
    clients.set(socket, { pushToken, timer });

    socket.send(
      JSON.stringify({
        type: "ack",
        message: `registered! push every ${INTERVAL_MS / 1000}s`,
      })
    );
  }
});

socket.on("close", () => {
  const client = clients.get(socket);
  if (client?.timer) clearInterval(client.timer);
  clients.delete(socket);
  console.log("client disconnected – timer cleared");
});

socket.on("error", (err) => {
  console.error("socket error:", err.message);
});
});

```

PROF

Your Tasks

TODO 1 — Write the `usePushNotifications` hook

File: `mobile/hooks/usePushNotifications.ts`

You did this in the tutorial. Apply that exact same knowledge here.

The hook lives outside the component so the logic stays clean and reusable.

The hook must return `{ expoPushToken, lastNotification }`.

Here's the skeleton — fill in every `// TODO` comment:

```

import { useState, useEffect, useRef } from "react";
import * as Notifications from "expo-notifications";
import * as Device from "expo-device";
import Constants from "expo-constants";
import { Platform } from "react-native";

// TODO: call Notifications.setNotificationHandler() here, outside the
hook.
// This controls how notifications appear when the app is already open.
// Use shouldShowBanner and shouldShowList

export function usePushNotifications() {
  const [expoPushToken, setExpoPushToken] = useState<string | null>
(null);
  const [lastNotification, setLastNotification] =
    useState<Notifications.Notification | null>(null);

  // TODO: declare two refs for the event subscriptions using useRef
  // so they survive re-renders without triggering them.
  // Type: useRef<Notifications.EventSubscription>()

  useEffect(() => {
    // TODO: call registerForPushNotificationsAsync() and store the
result
    // in expoPushToken if it's not null

    // TODO: set up addNotificationReceivedListener – store the
subscription
    // in your ref and update lastNotification state when it fires

    // TODO: set up addNotificationResponseReceivedListener – store in
the
    // other ref and console.log when the user taps the notification

    // TODO: return a cleanup function that calls .remove() on both refs
}, []);

    return { expoPushToken, lastNotification };
  }

  async function registerForPushNotificationsAsync(): Promise<string |
null> {
    // TODO: return null if not running on a real device (Device.isDevice)

    // TODO: check existing permission with getPermissionsAsync()
    // if not granted, request it with requestPermissionsAsync()
    // if still not granted after asking, return null

    // TODO: on Android, call setNotificationChannelAsync BEFORE
getExpoPushTokenAsync

```

```
// channel id: "default", importance:
Notifications.AndroidImportance.MAX
// (this ordering is required – the docs are explicit about it)

// TODO: get the projectId from Constants:
// Constants?.expoConfig?.extra?.eas?.projectId ??
Constants?.easConfig?.projectId
// return null if it's missing

// TODO: call getExpoPushTokenAsync({ projectId }) and return
token.data

return null; // remove this line once you've implemented the above
}
```

Things to pay attention:

- `setNotificationChannelAsync` must be called **before** `getExpoPushTokenAsync` on Android — wrong order means no token
- `shouldShowBanner` and `shouldShowList` are the current fields — `shouldShowAlert` still exists but is deprecated
- Both listener refs need to be cleaned up in the `useEffect` return, otherwise you get duplicate listeners on every re-render

TODO 2 — Connect to the server and send the push token

File: `mobile/app/index.tsx` → find `//` — `TODO 2`

The push token is the key element — the server can't reach the phone without it.

This `useEffect` runs whenever `expoPushToken` changes, so it fires once the hook resolves it.

1. Return early if `expoPushToken` is null — don't open the socket before the token is ready
2. `const socket = new WebSocket(WS_URL)` then `ws.current = socket`
3. `socket.onopen` → send the token as a JSON string:

```
{ "type": "register", "pushToken": "ExponentPushToken[...]" }
```

Then `setConnected(true)` and `addLog("connected – token sent to server")`

4. `socket.onmessage` → `parse event.data`; if `data.type === "ack"` → `addLog(\server: ${data.message})``
5. `socket.onclose` → `setConnected(false)`, `addLog("disconnected")`
6. `socket.onerror` → `addLog("connection error")`, `setConnected(false)`
7. Return `() => ws.current?.close()`

When it works, the log panel shows "connected — token sent to server" and the server terminal prints "push sent, receipt id: ..." within a few seconds — it sends the first notification immediately.

TODO 3 — Count incoming notifications

File: `mobile/app/index.tsx` → find `//` — **TODO 3**

`lastNotification` comes from your hook (the listener you set up in TODO 1).
It updates each time a push notification arrives.

1. Return early if `lastNotification` is null
2. `setNotifCount(prev => prev + 1)`
3. `const title = lastNotification.request.content.title ?? "no title"`
then `addLog(\got notification: "${title}")``

When it works the big counter ticks up each time a notification arrives.